

Quiz 1

601.471/671 NLP: Self-Supervised Models

Spring 2025

Johns Hopkins University

February 2025

Complete all questions.

Use additional paper if needed.

Time: 70 minutes.

Name of student: _____

Q1. Concepts, Foundations and Big Picture

22 points

For each of the following questions, select the most appropriate answer.

1. (1 point) What best describes Self-Supervised Learning?
- ☐ A model trained on a small labeled dataset to predict specific outcomes.
 - ☐ A model trained on large amounts of unlabeled data, learning predictive world models, tightly connected to real-world tasks.
 - ☐ A model that requires explicit human supervision to learn.
 - ☐ A form of supervised learning with less labeled data.

Solution: *The second one.*

2. (1 point) What is a key advantage of Self-Supervised Learning over Supervised Learning?
- ☐ It removes the need for large-scale labeled data.
 - ☐ It achieves perfect accuracy on all tasks.
 - ☐ It is less computationally expensive.
 - ☐ It requires no training.

Solution: *The first one.*

3. (1 point) What describes Language Modeling?
- ☐ Learning the probability distribution over sequences of words.
 - ☐ Predicting the previous word in a sentence given the next context.
 - ☐ Predicting the missing word in a sentence given prior and next context.
 - ☐ All of the above.

Solution: *All of the above.*

4. (1 point) What is the best description of “count-based language models”?
- ☐ Models that count the number of characters in a language.
 - ☐ Models that count the number of words in a document.
 - ☐ Models that predict word sequences.
 - ☐ Models that predict word sequences based on the frequency of occurrence in the given data.
 - ☐ Models that train on a specific word count to improve computational efficiency.

Solution: *The fourth one.*

5. (1 point) What is the sparsity problem in count-based language models?
- ☐ The problem of storing too much redundant information in language models.
 - ☐ The issue of too many zero or rare probability estimates due to unseen word sequences.
 - ☐ The difficulty in predicting numerical data rather than text.
 - ☐ The inability to model multi-word expressions.

Solution: *The 2nd one.*

6. (1 point) True or false: A bigram language model with vocabulary size V can produce any sentence of length n with non-zero probability, regardless of how it was trained. ☐ True ☐ False

Solution: *False*

7. (1 point) True or false: Given *any* corpus with vocab size V , there *always* exists a language model for the corpus that has a perplexity of V . ☐ True ☐ False

Solution: *True*

8. (1 point) True or false: Any function approximated by a three-hidden-layer network can also be approximated by a two-hidden-layer network, given sufficient width. ☐ True ☐ False

Solution: *True*

9. (1 point) You evaluate a classifier on 20 instances (inputs). The classifier assigns equal probability to each of 10 classes on each instance. What is the cross-entropy loss when *averaged* over these instances?
- ☐ $-\log(10)$, ☐ $-\log(20)$, ☐ $-0.1 \log(10)$, ☐ $-0.1 \log(20)$,
☐ $-0.2 \log(10)$, ☐ $-\log(0.1)$, ☐ $-\log(0.2)$, ☐ $-10 \log(0.1)$,
☐ $-20 \log(0.1)$, ☐ $-20 \log(0.2)$, ☐ $-10 \log(0.2)$.

Solution: $-\log(0.1)$. Notice that it's "averaged" not summed (and hence, $-20 \log(0.1)$ is incorrect).

10. (1 point) Which statement about "dropout" is correct?
- ☐ It prevents overfitting by randomly deactivating neurons during training.
☐ It speeds up training significantly.
☐ It only works on convolutional networks.
☐ It is always applied at inference time.

Solution: The first one.

11. (1 point) Increasing the temperature parameter in the Softmax function makes the output distribution more peaked, increasing confidence in the highest probability class. ☐ True ☐ False

Solution: False

12. (1 point) How does "normalization" help train deep networks?
- ☐ It improves convergence.
☐ It prevents overfitting entirely.
☐ It ensures gradients never vanish.
☐ It replaces dropout.

Solution: The first one.

13. (2 points) What are the limitations of the Sigmoid activation function? Select all that apply.
- ☐ Does not allow multiple outputs.
☐ Can cause vanishing gradients during backpropagation.
☐ Cannot be used in binary classification.
☐ Not zero-centered output.

Solution: 2nd and 4th.

14. (1 point) True or false: During backpropagation, when the gradient flows backward through the Sigmoid or Tanh non-linearities, it cannot change the sign. ☐ True ☐ False

Solution: True, the Local Gradient is always positive

15. (1 point) True or false: Suppose a fully connected neural network is initialized with identical weights for all neurons at each layer. Even if the weights are not zero, the network can still learn distinct features because the input data is non-uniform. ☐ True ☐ False

Solution: False

16. (1 point) You're training your neural net and expect it to give you probabilities. But to your surprise, it produces numbers that are > 1 or < 0 . Which step in the training pipeline is most likely to be incorrectly implemented?
- ☐ Initializing weights from a normal distribution.
☐ Using a dropout layer after each hidden layer.
☐ Normalizing outputs with a Softmax function in the final layer.
☐ Reducing the learning rate gradually during training.

Solution: Third choice

17. (1 point) What is an incorrect statement about backpropagation in neural networks?
- ☐ Backpropagation uses gradient descent to minimize the loss function.

- ☐ Backpropagation adjusts the weights in the neural network based on the output error.
- ☐ Backpropa is a sequential process and its complexity is a quadratic function of the number of layers.
- ☐ None of the above.

Solution: *The 3rd one.*

18. (1 point) What is a Jacobian matrix, and how is it related to neural networks?
- ☐ A matrix that represents the weights of a neural network.
 - ☐ A matrix that is used to generate random numbers for initializing weights in a neural network.
 - ☐ A matrix that is used to pre-process data before feeding it into a neural network.
 - ☐ A matrix containing a function's derivatives and is used to update the weights in a neural network during backprop.
 - ☐ All of the above.

Solution: *A Jacobian matrix is a matrix based on the partial differentiation of a function, but first-order partial derivatives. It tells us how local perturbations the neural network input would impact the output. The correct answer is the 4th option.*

19. (1 point) What is the learning rate's impact on the gradient descent process?
- ☐ A high learning rate may lead to quick convergence but risk overshooting the minimum
 - ☐ A low learning rate results in slower convergence but risk overshooting the minimum
 - ☐ The learning rate has no impact on the convergence of the gradient descent algorithm
 - ☐ High learning rates completely eliminate the risk of overshooting.
 - ☐ None of the above.

Solution: *The first one.*

20. (1 point) In what situation would you intentionally want to overfit a model, and why?
- ☐ When the dataset is very large and diverse, and you want to capture every individual data point.
 - ☐ When the model is subjected to high noise levels, and you want to increase its robustness.
 - ☐ Never, overfitting is always undesirable.
 - ☐ When the dataset is small and noisy, you might want to overfit to reduce bias.
 - ☐ When you want the model to perfectly predict the training data, make sure that it has enough capacity and that the training works as expected.

Solution: *The last one*

21. (1 point) What typically causes vanishing or exploding gradients in a neural network?
- ☐ Not using appropriate activation functions, such as ReLU.
 - ☐ The deep architecture of the network and the use of certain activation functions like sigmoid or tanh.
 - ☐ Specific weights and bias strategies, like zero initialization.
 - ☐ Overfitting or underfitting on the provided training data.
 - ☐ None of the above

Solution: *2nd choice*

Q2. Math Foundation

8 points

1. (4 points) Compute the Jacobian matrix for the following function: $f(x, y, z) = [x^2 + y, xy, z^2/y]$. Then instantiate it at $(x, y, z) = (1, 1/2, -1)$.

Solution: The Jacobian matrix:

$$J_f(x, y, z) = \begin{bmatrix} 2x & 1 & 0 \\ y & x & 0 \\ 0 & -\frac{z^2}{y^2} & \frac{2z}{y} \end{bmatrix}$$

The instantiation:

$$J_f(x = 1, y = 1/2, z = -1) = \begin{bmatrix} 2 & 1 & 0 \\ 1/2 & 1 & 0 \\ 0 & -4 & -4 \end{bmatrix}$$

2. (4 points) Consider the function: $J(w) = (w - 3)^2 + 4$. Perform **one** step of gradient descent with learning rate $\eta = 0.1$, starting from $w_0 = 6$.

Solution: The gradient descent update rule is:

$$w_{t+1} = w_t - \eta \cdot \nabla J(w_t)$$

where:

$$\nabla J(w) = \frac{d}{dw} ((w - 3)^2 + 4) = 2(w - 3)$$

Gradient at $w_0 = 6$

$$\nabla J(6) = 2(6 - 3) = 6$$

$$w_1 = 6 - 0.1 \times 6 = 6 - 0.6 = 5.4$$

Q3. Language Modeling and N-Gram Models

15 points

1. (4 points) Consider the following vocabulary, $V = \{\text{BOS, EOS, The, dog, cat, ran, barked, jumped, Tianjian}\}$ where BOS and EOS are the auxiliary token indicating the beginning and the end of a sentence, respectively. Consider the following training data:

```
BOS who EOS
BOS The dog ran EOS
BOS The cat ran EOS
BOS The dog barked EOS
BOS Tianjian ran EOS
```

n -gram	Probability
$P(\text{The} \text{BOS})$	
$P(\text{Tianjian} \text{BOS})$	
$P(\text{cat} \text{The})$	
$P(\text{dog} \text{The})$	
$P(\text{jumped} \text{cat})$	

n -gram	Probability
$P(\text{barked} \text{cat})$	
$P(\text{ran} \text{cat})$	
$P(\text{ran} \text{dog})$	
$P(\text{EOS} \text{ran})$	
$P(\text{EOS} \text{jumped})$	

Compute the **bigram** probabilities. If you face any ill-defined 0/0 division, replace it with a small probability $\epsilon = 0.01$. Use the same logic for any potential zero probabilities (zero divided by a non-zero value). Please show your work (the intermediate calculations/counting) in the box below. Write the final answers (probabilities) in the above table.

Solution: First we count unigram counts: BOS: 5, Tianjian: 1, The: 3, who: 1, cat: 1, dog: 2, ran: 3, barked: 1. Then we count bigram counts: BOS The: 3, BOS Tianjian: 1, The cat: 1, The dog: 2, cat jumped: 0, cat barked: 0, cat ran: 1, dog ran: 1, ran EOS: 3, jumped EOS: 0, This gives us the following table:

<i>n</i> -gram	Probability
$P(\text{The} \text{BOS})$	3/5
$P(\text{Tianjian} \text{BOS})$	1/5
$P(\text{cat} \text{The})$	1/3
$P(\text{dog} \text{The})$	2/3
$P(\text{jumped} \text{cat})$	0/1

<i>n</i> -gram	Probability
$P(\text{barked} \text{cat})$	0/1 $\rightarrow 0.01$
$P(\text{ran} \text{cat})$	1/1
$P(\text{ran} \text{dog})$	1/2
$P(\text{EOS} \text{ran})$	3/3
$P(\text{EOS} \text{jumped})$	0/0 (ill-defined) $\rightarrow 0.01$

2. (4 points) Given this bigram language model, compute the probability of $s_1 := \text{"BOS The dog ran EOS"}$ and $s_2 := \text{"BOS The cat jumped EOS"}$. No need to simplify the fraction. Hint: $P(s_1) = P(\text{BOS}) \times P(\text{The}|\text{BOS}) \times \dots$ and since all sentences start with BOS, you can assume $P(\text{BOS}) = 1$.

Solution:

$$P(s_1) = P(\text{BOS})P(\text{The}|\text{BOS})P(\text{dog}|\text{The})P(\text{ran}|\text{dog})P(\text{EOS}|\text{ran}) = 1 \times 3/5 \times 2/3 \times 1/2 \times 3/3 = 0.2$$

$$\begin{aligned} P(s_2) &= P(\text{BOS})P(\text{The}|\text{BOS})P(\text{cat}|\text{The})P(\text{jumped}|\text{cat})P(\text{EOS}|\text{jumped}) \\ &= 1 \times 3/5 \times 1/3 \times 0/1 (\rightarrow 0.01) \times 0/0 (\rightarrow 0.01) = 0.00002 \end{aligned}$$

3. (4 points) Compute the perplexity of the previous two sentences using your bigram language model. When computing the sentence length, only using the bigrams (i.e., ignore BOS in the sentence length calculations). Use log with base 2. No need to simplify the fractions or logarithms.

Solution:

$$H(s_1) = -\frac{1}{4}[\log_2 3/5 + \log_2 2/3 + \log_2 1/2 + \log_2 3/3] = 0.58 \Rightarrow \text{ppl}(w_1, \dots, w_n) = 2^H = 1.49$$

$$H(s_2) = -\frac{1}{4}[\log_2 3/5 + \log_2 1/3 + \log_2 0.01 + \log_2 0.01] = 3.24 \Rightarrow \text{ppl}(w_1, \dots, w_n) = 2^H = 9.447$$

4. (3 points) The probability of which sentence is higher (and why)? The perplexity of which sentence is higher (and why)?

Solution: The probability of the first sentence is higher because it contains unobserved patterns. The second sentence has a higher perplexity, as PPL measures uncertainty—sentences with lower probabilities result in higher PPL values.

Q4. PyTorch Understanding

28 points

1. Consider the following PyTorch code for a simple neural net for binary classification.

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class FeedforwardNN(nn.Module):
6     def __init__(self, input_size):
7         super(FeedforwardNN, self).__init__()
8         self.hidden = nn.Linear(input_size, 5, bias=False)
```

```

9         self.output = nn.Linear(5, 2, bias=False)
10
11     def forward(self, x):
12         x = F.relu(self.hidden(x))
13         x = self.output(x)
14         return x

```

- (a) (4 points) What is the purpose of `F.relu(self.hidden(x))` and why are they needed?

Solution: The function `F.relu(self.hidden(x))` applies the ReLU (Rectified Linear Unit) activation function to the output of the hidden layer. This introduces non-linearity, allowing the network to learn complex patterns beyond linear transformations.

- (b) (4 points) What is the primary purpose of the `torch.nn.Module` class? (2-3 sentences)

Solution: The `torch.nn.Module` class provides a base for all neural network modules in PyTorch. It helps in managing layers, parameters, and defining the forward computation of the network.

- (c) (4 points) If `input_size = 3`, how many trainable parameters does this network have? Hint: The linear layers here have no bias term.

Solution: First linear layer (`self.hidden = nn.Linear(input_size, 5)`) weights: $3 \times 5 = 15$. Second linear layer (`self.output = nn.Linear(5, 2)`) weights $5 \times 2 = 10$. So in total: 25.

2. The remainder of the code is provided below to compute the forward pass and return the predicted probabilities.

```

1 import torch.optim as optim
2 batch_size = 4
3 input_size = 3
4 model = FeedforwardNN(input_size=input_size)
5 criterion = nn.CrossEntropyLoss(reduction="mean")
6 optimizer = optim.SGD(model.parameters(), lr=0.01)
7 input_tensor = torch.rand((batch_size, input_size))
8 target_tensor = torch.randint(0, 2, (batch_size,)) # creates a tensor of length batch size.
              Each element is of random integers between 0 and 1
9 for epoch in range(10):
10     optimizer.zero_grad() # Step 1
11     output_logits = model(input_tensor) # Step 2
12     output_probs = _____ # Step3: Convert logits to probabilities
13     loss = _____ # Step 4: Compute loss with cross-entropy
14     loss.backward() # Step 5
15     _____ # Step 6: Update weights

```

- (a) (4 points) Fill in the missing line to compute Step 3 (probabilities from the output logits), Step 4 (compute loss) and Step 6 (update weights). Write your answer in the following box.

Solution: Step 3: `output_probs = F.softmax(output_logits, dim=1)`
 Step 4: `loss = torch.nn.functional.cross_entropy(output_logits, target_tensor)`
 Step 6: `optimizer.step()`

- (b) (4 points) What is the output tensor (`output_logits`) shape if the shape of `input_tensor` is `(batch_size, 3)`? How about dimensions (`output_probs`)? How about dimensions (`loss`)?

Solution: Since `input_tensor` has shape `(batch_size, 3)`, it passes through `Linear(3, 5)` → `ReLU` → `Linear(5, 2)`, resulting in an output of shape `(batch_size, 2)`. `Softmax` does not change shape, it just normalizes logits into probabilities. The loss's shape is scalar.

- (c) (4 points) Explain what happens during `backward()` (1-2 sentences).

Solution: Calling `loss.backward()` computes the gradients of the loss function with respect to each parameter in the model using backpropagation.

(d) (4 points) Why do we call `optimizer.zero_grad()` before `loss.backward()`? (1-2 sent.)

Solution: Gradients accumulate by default in PyTorch, meaning that if we don't reset them, they add up across multiple `backward()` calls. Calling `optimizer.zero_grad()` ensures that each batch starts with a clean slate, preventing unwanted gradient accumulation.

Q5. Neural Nets and Backpropagation

27 points

1. (4 points) The **softmax** function of a vector $\mathbf{z} \in \mathbb{R}^d$ is defined as $\sigma(\mathbf{z}) = [\sigma_1(z) \dots \sigma_d(z)]$ where $\sigma_i(\mathbf{z}) = \frac{e^{z_i}}{\sum_{j=1}^d e^{z_j}}$. Compute the softmax outputs for $\mathbf{z} = [2, 1, 0]$. No need to simplify the fraction.

Solution:

$$\left[\frac{e^2}{1 + e^1 + e^2}, \frac{e^1}{1 + e^1 + e^2}, \frac{1}{1 + e^1 + e^2} \right]$$

2. (4 points) Now consider the softmax of $\mathbf{z} = [10002, 10001, 10000]$. How does this result compare to the previous question.

Solution: It's the same since softmax is shift invariant.

3. (4 points) Let X be a vector of size n . Let $X' = \text{softmax}(X)$. What is $\sum_{x' \in X'} x'$?

Solution: 1

4. (4 points) Suppose $z \in \mathbb{R}$. Also let σ denote the **sigmoid** function: $\sigma(z) = 1/(1 + e^{-z})$. Prove that the gradient of the sigmoid function: $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$.

Solution: So assume that $\sigma(z) = (1 + e^{-z})^{-1}$. From basic algebra we know:

$$\frac{d\sigma(z)}{dz} = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

We can rearrange to express e^{-z} as:

$$e^{-z} = \frac{1 - \sigma(z)}{\sigma(z)}$$

Substituting into our derivative:

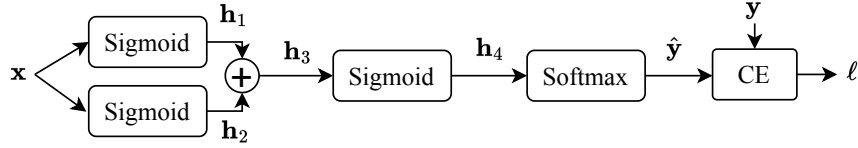
$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

5. (4 points) Assume that $\mathbf{z} = [z_1, \dots, z_d] \in \mathbb{R}^d$. Then define the sigmoid of a vector variable as an element-wise sigmoid: $\sigma(\mathbf{z}) = [\sigma(z_1), \dots, \sigma(z_d)]$. What is the Jacobian of $\mathbf{z}$?

Solution:

$$J_{\sigma}(\mathbf{z}) = \begin{bmatrix} \sigma(x_1)(1 - \sigma(x_1)) & 0 & 0 & \dots & 0 \\ 0 & \sigma(x_2)(1 - \sigma(x_2)) & 0 & \dots & 0 \\ 0 & 0 & \sigma(x_3)(1 - \sigma(x_3)) & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma(x_n)(1 - \sigma(x_n)) \end{bmatrix}$$

6. (7 points) In the following neural network, compute the gradient $\frac{\partial \ell}{\partial \mathbf{x}}$.



Input: $\mathbf{x} \in \mathbb{R}^d$, Parameters: $\mathbf{W}_{1,2,3} \in \mathbb{R}^{d \times d}$, Output prediction: $\hat{\mathbf{y}} \in \mathbb{R}^d$

$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x}, \quad \mathbf{z}_2 = \mathbf{W}_2 \mathbf{x}, \quad \mathbf{h}_1 = \text{Sigmoid}(\mathbf{z}_1), \quad \mathbf{h}_2 = \text{Sigmoid}(\mathbf{z}_2), \quad \mathbf{h}_3 = \mathbf{h}_1 + \mathbf{h}_2,$$

$$\mathbf{z}_3 = \mathbf{W}_3 \mathbf{h}_3, \quad \mathbf{h}_4 = \text{Sigmoid}(\mathbf{z}_3), \quad \hat{\mathbf{y}} = \text{Softmax}(\mathbf{h}_4), \quad \ell = \text{CE}(\mathbf{y}, \hat{\mathbf{y}}),$$

You are allowed (and highly encouraged) to use variables to represent intermediate gradients.

Hint 1: Compute these gradients in order to build up your answer: $\frac{\partial \ell}{\partial \mathbf{h}_4}, \frac{\partial \ell}{\partial \mathbf{h}_3}, \frac{\partial \ell}{\partial \mathbf{z}_3}, \frac{\partial \ell}{\partial \mathbf{h}_1}, \frac{\partial \ell}{\partial \mathbf{z}_1}, \frac{\partial \ell}{\partial \mathbf{h}_2}, \frac{\partial \ell}{\partial \mathbf{z}_2}, \frac{\partial \ell}{\partial \mathbf{x}}$. Show your work so we can give partial credit!

Hint 2: Recall that $\text{downstream} = \text{upstream} * \text{local}$. Throughout your derivations, clearly denote upstream and local gradients.

Hint 3: Assume that $\frac{\partial \ell}{\partial \mathbf{h}_4} = \hat{\mathbf{y}} - \mathbf{y}$. You will prove this in HW4!

Solution: Let's define the following *local* gradients;

$$l_1 = \frac{\partial \mathbf{h}_4}{\partial \mathbf{z}_3} = J_{h_4}(\mathbf{z}_3) \text{ (from question 5)}$$

$$l_2 = \frac{\partial \mathbf{z}_3}{\partial \mathbf{h}_3} = \mathbf{W}_3$$

$$l_3 = \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} = 1, l_4 = \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_1} = 1$$

$$l_5 = \frac{\partial \mathbf{h}_1}{\partial \mathbf{z}_1} = J_{h_1}(\mathbf{z}_1) \text{ (from question 5)}, l_6 = \frac{\partial \mathbf{h}_2}{\partial \mathbf{z}_2} = J_{h_2}(\mathbf{z}_2) \text{ (from question 5)}$$

$$l_6 = \frac{\partial \mathbf{z}_1}{\partial \mathbf{x}} = \mathbf{W}_1, l_7 = \frac{\partial \mathbf{z}_2}{\partial \mathbf{x}} = \mathbf{W}_2$$

Now, we get to combine local gradients with upstream gradients. From hint3, we see the following upstream gradient: $\delta_1 = \frac{\partial \ell}{\partial \mathbf{h}_4} = \hat{\mathbf{h}}_4 - \mathbf{y}$.

Now, we can propagate it to the earlier layers:

$$\delta_2 = \frac{\partial \ell}{\partial \mathbf{z}_3} = \frac{\partial \ell}{\partial \mathbf{h}_4} \frac{\partial \mathbf{h}_4}{\partial \mathbf{z}_3} = \delta_1 l_1$$

$$\delta_3 = \frac{\partial \ell}{\partial \mathbf{h}_3} = \frac{\partial \ell}{\partial \mathbf{z}_3} \frac{\partial \mathbf{z}_3}{\partial \mathbf{h}_3} = \delta_2 \mathbf{W}_3$$

$$\frac{\partial \ell}{\partial \mathbf{h}_2} = \frac{\partial \ell}{\partial \mathbf{h}_3} \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} = \delta_3, \frac{\partial \ell}{\partial \mathbf{h}_1} = \frac{\partial \ell}{\partial \mathbf{h}_3} \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_1} = \delta_3$$

$$\delta_4 = \frac{\partial \ell}{\partial \mathbf{z}_1} = \frac{\partial \ell}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{z}_1} = \delta_3 l_5, \delta_5 = \frac{\partial \ell}{\partial \mathbf{z}_2} = \frac{\partial \ell}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{z}_2} = \delta_3 l_6$$

$$\frac{\partial \ell}{\partial \mathbf{x}} = \frac{\partial \ell}{\partial \mathbf{z}_1} \frac{\partial \mathbf{z}_1}{\partial \mathbf{x}} + \frac{\partial \ell}{\partial \mathbf{z}_2} \frac{\partial \mathbf{z}_2}{\partial \mathbf{x}} = \delta_4 \mathbf{W}_1 + \delta_5 \mathbf{W}_2$$